

explain me os module

Python os Module — Detailed Explanation

The Python **os** **module** lets you interact with the **operating system** from Python.

You can use it to:

- Work with files and folders
- Create directories
- Delete files/directories
- Rename and move files
- Check whether files exist
- Get file information
- Work with environment variables
- Get the current working directory
- Navigate between directories
- Build file paths
- Run operating-system commands

For your **data engineering / Spark Structured Streaming practice**, **os** is especially useful for **creating JSON files automatically and saving them into directories**.

1. Import os

```
</> Python
```

[▶ Run](#)

```
import os
```

After importing it, you can use:

```
</> Python
```

[▶ Run](#)

```
os.function()
```

For example:

```
</> Python

import os

print(os.getcwd())
```

[Run](#)

2. Current Working Directory

The **current working directory (CWD)** is the directory where Python is currently working.

```
</> Python

import os

print(os.getcwd())
```

[Run](#)

Example output:

```
/home/youssef/projects
```



Think of it as:

```
Python is currently here
      ↓
/home/youssef/projects
```



3. Change Directory — `os.chdir()`

You can change the current working directory.

```
</> Python

import os

os.chdir("/home/youssef/data")

print(os.getcwd())
```

[Run](#)

Now Python is working inside:

```
/home/youssef/data
```



You can also use:

```
</> Python  
  
os.chdir("data")
```



▶ Run

if `data` is relative to the current directory.

4. List Files and Folders — `os.listdir()`

This is one of the most useful functions.

```
</> Python  
  
import os  
  
print(os.listdir())
```



▶ Run

Example:

```
['file1.txt', 'file2.json', 'data', 'images']
```



You can specify a directory:

```
</> Python  
  
print(os.listdir("/home/youssef/data"))
```



▶ Run

Example:

```
['customers.csv', 'orders.json', 'employees.csv']
```



5. Check If a File or Directory Exists

Use:

```
</> Python  
  
os.path.exists()
```



▶ Run

Example:

</> Python



Run

```
import os

if os.path.exists("data"):
    print("data exists")
else:
    print("data does not exist")
```

For a file:

</> Python



Run

```
if os.path.exists("data.json"):
    print("File exists")
```

6. Check Whether Something Is a File

Use:

</> Python



Run

```
os.path.isfile()
```

Example:

</> Python



Run

```
import os

if os.path.isfile("data.json"):
    print("This is a file")
```

7. Check Whether Something Is a Directory

Use:

</> Python



Run

```
os.path.isdir()
```

Example:

</> Python



Run

```
if os.path.isdir("data"):
    print("This is a directory")
```

So:

</> Python



Run

```
os.path.exists()
```

means:

Does it exist?

While:

</> Python



Run

```
os.path.isfile()
```

means:

Is it a file?

And:

</> Python



Run

```
os.path.isdir()
```

means:

Is it a directory?

8. Create a Directory — `os.mkdir()`

</> Python



Run

```
import os

os.mkdir("data")
```

This creates:

```
current_directory/  
└─ data/
```



Important

If `data` already exists:

```
</> Python  
  
os.mkdir("data")
```



▶ Run

will produce an error.

9. Create Multiple Directories — `os.makedirs()`

Suppose you want:

```
project/  
└─ data/  
    └─ json/  
        └─ input/
```



You can do:

```
</> Python  
  
import os  
  
os.makedirs("data/json/input")
```



▶ Run

This creates all missing directories.

You can also use:

```
</> Python  
  
os.makedirs("data/json/input", exist_ok=True)
```



▶ Run

This is extremely useful.

`exist_ok=True` means:

Don't raise an error if the directory already exists.

For example:

```
</> Python
os.makedirs("data/json", exist_ok=True)
```



▶ Run

You can run this repeatedly without errors.

10. Remove a Directory — `os.rmdir()`

```
</> Python
import os

os.rmdir("data")
```



▶ Run

This removes an **empty** directory.

If the directory contains files, you'll get an error.

11. Remove a File — `os.remove()`

```
</> Python
import os

os.remove("data.json")
```



▶ Run

This deletes:

```
data.json
```



Be careful: deletion is permanent unless you have another copy.

12. Rename a File

Use:

```
</> Python
```



▶ Run

```
os.rename()
```

Example:

```
</> Python
```



▶ Run

```
import os
```

```
os.rename("old.json", "new.json")
```

Before:

```
old.json
```



After:

```
new.json
```



13. Move a File

`os.rename()` can also move files.

For example:

```
</> Python
```



▶ Run

```
os.rename(  
    "data.json",  
    "archive/data.json"  
)
```

This moves:

```
data.json
```



into:

```
archive/  
└─ data.json
```



However, for more advanced file operations, Python's `shutil` module is often better.

14. File Paths — `os.path.join()`

This is **very important**.

Instead of manually writing:

```
</> Python
path = "data/" + "file.json"
```



▶ Run

use:

```
</> Python
path = os.path.join("data", "file.json")
```



▶ Run

Result on Linux:

```
data/file.json
```



On Windows it automatically handles the appropriate path separator.

For example:

```
</> Python
import os

folder = "data"
filename = "events.json"

path = os.path.join(folder, filename)

print(path)
```



▶ Run

Output:

```
data/events.json
```



15. Why `os.path.join()` Is Better

Avoid:

</> Python



Run

```
path = folder + "/" + filename
```

Prefer:

</> Python



Run

```
path = os.path.join(folder, filename)
```

Because Python handles the operating system's path format.

For example:

Linux:

data/file.json



Windows:

data\file.json

16. Get the File Name From a Path

Use:

</> Python



Run

```
os.path.basename()
```

Example:

</> Python



Run

```
import os

path = "/home/youssef/data/events.json"

print(os.path.basename(path))
```

Output:

events.json



17. Get the Directory From a Path

Use:

```
</> Python
```



▶ Run

```
os.path.dirname()
```

Example:

```
</> Python
```



▶ Run

```
path = "/home/youssef/data/events.json"

print(os.path.dirname(path))
```

Output:

```
/home/youssef/data
```



So:

```
/home/youssef/data/events.json
```



```
└──────────┘
```

dirname

```
/home/youssef/data/events.json
```

```
└──┘
```

basename

18. Get File Extension

You can use:

```
</> Python
```



▶ Run

```
os.path.splitext()
```

Example:

```
</> Python
```



▶ Run

```
import os

filename = "events.json"

name, extension = os.path.splitext(filename)

print(name)
print(extension)
```

Output:

```
events
.json
```



Another example:

```
</> Python

filename = "customers.csv"

name, extension = os.path.splitext(filename)

print(name)
print(extension)
```



▶ Run

Output:

```
customers
.csv
```



19. File Size — `os.path.getsize()`

```
</> Python

import os

size = os.path.getsize("data.json")

print(size)
```



▶ Run

The result is in **bytes**.

For example:

```
1024
```



means:

```
1024 bytes = 1 KB
```



approximately.

20. Environment Variables

`os` can access environment variables.

Use:

```
</> Python  
  
os.environ
```



▶ Run

Example:

```
</> Python  
  
import os  
  
print(os.environ)
```



▶ Run

This can contain things such as:

```
PATH  
HOME  
USER  
JAVA_HOME  
SPARK_HOME
```



21. Get One Environment Variable

Use:

```
</> Python  
  
os.getenv()
```



▶ Run

Example:

</> Python



▶ Run

```
import os

home = os.getenv("HOME")

print(home)
```

On Linux you might get:

/home/youssef



Another example:

</> Python



▶ Run

```
java_home = os.getenv("JAVA_HOME")

print(java_home)
```

22. Why Environment Variables Matter in Data Engineering

You may have:

JAVA_HOME
HADOOP_HOME
SPARK_HOME
PATH



For example:

</> Python



▶ Run

```
import os

print(os.getenv("JAVA_HOME"))
print(os.getenv("HADOOP_HOME"))
print(os.getenv("SPARK_HOME"))
```

This can help you check your environment.

23. Set an Environment Variable

You can set one for the current Python process:

</> Python



▶ Run

```
import os

os.environ["MY_VARIABLE"] = "hello"

print(os.getenv("MY_VARIABLE"))
```

Output:

hello



Important:

This normally affects the **current Python process and processes it starts**; it does not permanently modify your shell configuration.

24. `os.walk()` — Very Important

`os.walk()` lets you recursively walk through directories.

Suppose:

```
data/
├─ file1.json
├─ file2.json
├─ customers/
│   ├─ customer1.json
│   └─ customer2.json
└─ orders/
    ├─ order1.json
    └─ order2.json
```



You can do:

</> Python



▶ Run

```
import os

for root, dirs, files in os.walk("data"):
    print("ROOT:", root)
    print("DIRECTORIES:", dirs)
    print("FILES:", files)
```

It will go through the entire directory tree.

25. Find All JSON Files

This is useful for data engineering.

```
</> Python   Run

import os

for root, dirs, files in os.walk("data"):

    for file in files:

        if file.endswith(".json"):
            print(os.path.join(root, file))
```

Possible output:

```
data/file1.json
data/file2.json
data/customers/customer1.json
data/customers/customer2.json
data/orders/order1.json
data/orders/order2.json 
```

26. os.system()

You can execute an operating-system command.

Example:

```
</> Python   Run

import os

os.system("ls")
```

On Linux this executes:

```
</> Bash
```



```
ls
```

You could also do:

```
</> Python
```



▶ Run

```
os.system("pwd")
```

which executes:

```
</> Bash
```



```
pwd
```

But

For serious Python applications, `subprocess` is generally preferred over `os.system()` because it provides better control over processes, arguments, errors, and output.

27. `os.name`

You can determine the operating-system family.

```
</> Python
```



▶ Run

```
import os
```

```
print(os.name)
```

On Linux:

```
posix
```



On Windows:

```
nt
```



28. os.getpid()

You can get the current process ID.

Python



Run

```
import os

print(os.getpid())
```

Example:

18342



This is the ID of the Python process.

29. Practical Example: Create a Data Directory

For your streaming practice, you might do:

Python



Run

```
import os

data_dir = "streaming_data"

os.makedirs(data_dir, exist_ok=True)

print("Directory created:", data_dir)
```

Result:

streaming_data/



30. Practical Example: Create JSON Files

Suppose you want to create:

```
streaming_data/  
├─ event_1.json  
├─ event_2.json  
├─ event_3.json  
└─ event_4.json
```



You can use `os` with Python's `json` module:

</> Python



▶ Run

```
import os  
import json  
  
data_dir = "streaming_data"  
  
os.makedirs(data_dir, exist_ok=True)  
  
event = {  
    "id": 1,  
    "product": "Laptop",  
    "price": 1000,  
    "quantity": 2  
}  
  
file_path = os.path.join(data_dir, "event_1.json")  
  
with open(file_path, "w") as file:  
    json.dump(event, file)  
  
print("Created:", file_path)
```

Now:

```
streaming_data/  
└─ event_1.json
```



contains:

</> JSON



```
{  
    "id": 1,  
    "product": "Laptop",  
    "price": 1000,  
    "quantity": 2  
}
```

31. Create Many JSON Files

This is particularly useful for **Spark Structured Streaming file sources**.

Python



Run

```
import os
import json
import time

data_dir = "streaming_data"

os.makedirs(data_dir, exist_ok=True)

for i in range(1, 6):

    event = {
        "id": i,
        "product": f"Product_{i}",
        "price": i * 100,
        "quantity": i
    }

    file_path = os.path.join(
        data_dir,
        f"event_{i}.json"
    )

    with open(file_path, "w") as file:
        json.dump(event, file)

    print("Created:", file_path)

    time.sleep(2)
```

You get:

```
streaming_data/
├─ event_1.json
├─ event_2.json
├─ event_3.json
├─ event_4.json
└─ event_5.json
```



This is a very good exercise for your Structured Streaming practice.

32. os.path Functions You Should Know

For your level, focus heavily on these:

Function	Purpose
<code>os.getcwd()</code>	Get current directory
<code>os.chdir()</code>	Change directory
<code>os.listdir()</code>	List files/directories
<code>os.mkdir()</code>	Create one directory
<code>os.makedirs()</code>	Create directories recursively
<code>os.remove()</code>	Delete a file
<code>os.rmdir()</code>	Delete an empty directory
<code>os.rename()</code>	Rename/move
<code>os.path.exists()</code>	Check existence
<code>os.path.isfile()</code>	Check if file
<code>os.path.isdir()</code>	Check if directory
<code>os.path.join()</code>	Build paths
<code>os.path.basename()</code>	Get filename
<code>os.path.dirname()</code>	Get directory
<code>os.path.splitext()</code>	Split filename/extension
<code>os.path.getsize()</code>	Get file size
<code>os.walk()</code>	Recursively walk directories
<code>os.getenv()</code>	Read environment variable
<code>os.environ</code>	Access environment variables

33. The Most Important Concept for You

Since you're learning Python for **Spark Structured Streaming**, think of `os` like this:

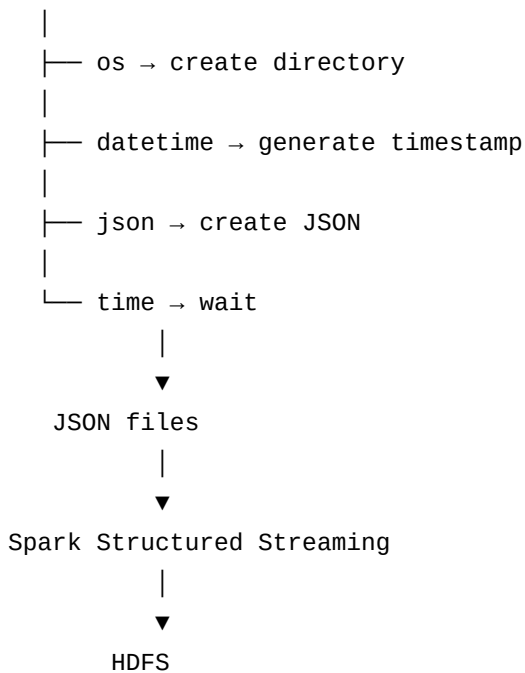
```
Python
├── os
│   ├── Create folders
│   ├── Find files
│   ├── Check files
│   ├── Build paths
│   ├── Rename files
│   ├── Delete files
│   └── Work with environment
```

Then combine it with:

```
os
+
json
+
datetime
+
time
```

to build a simple streaming-data generator:

Python script



What I recommend you learn next

For your current Spark Streaming goal, learn these four together:

`os` → `json` → `datetime` → `time`

Then you can build a Python program that continuously generates JSON event files, which Spark Structured Streaming reads automatically.



ChatGPT can make mistakes. Check important info.